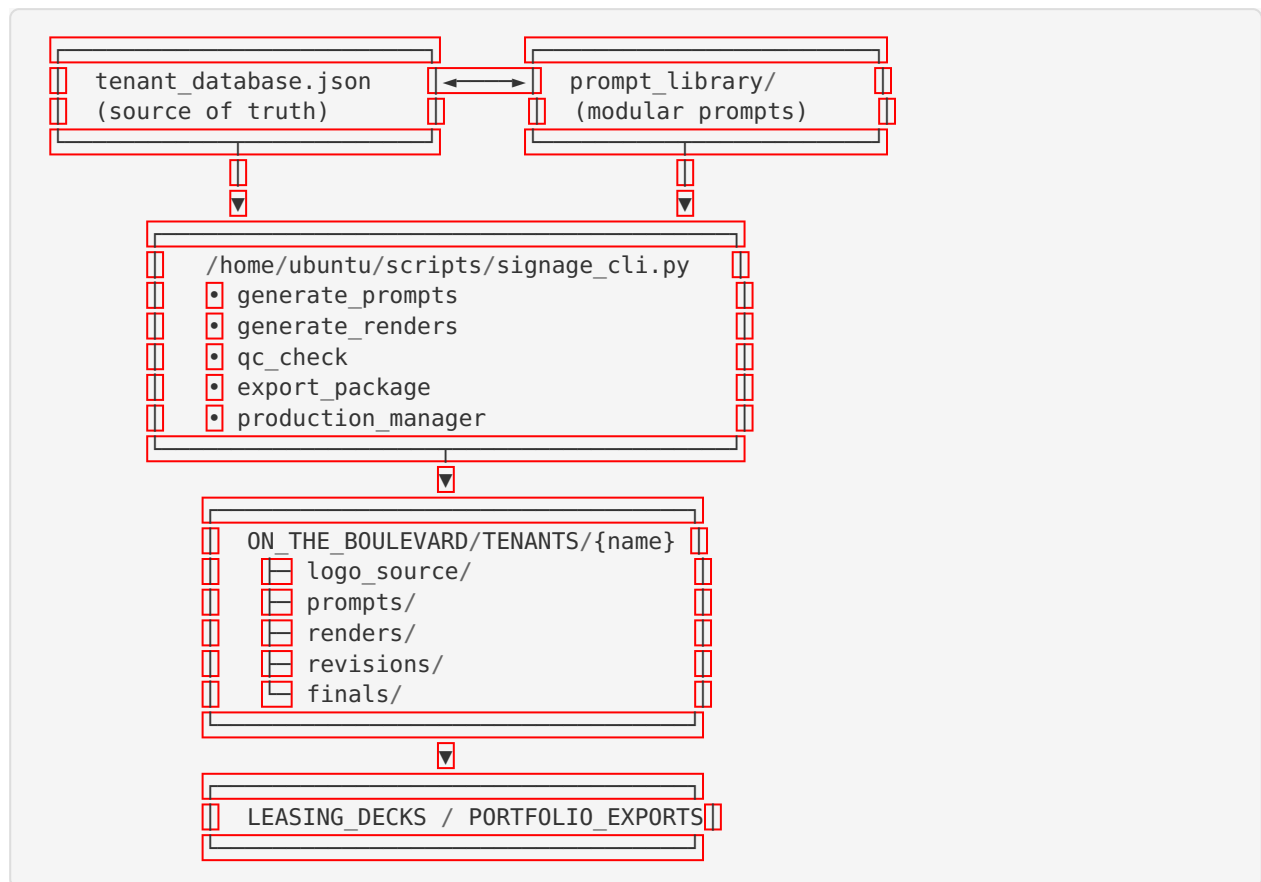


On The Boulevard — Operator Guide

A complete, non-technical walkthrough of the tenant signage rendering system.
If you can copy a file and run one command, you can operate this system.

1. System Overview

1.1 Architecture (high level)



1.2 Key components

Component	Path	Role
Tenant DB	/home/ubuntu/tenant_database.json	Tenant metadata, illumination, logo path
Prompt library	/home/ubuntu/prompt_library/	Layered prompt fragments (global → category → scene → tenant)
Scripts	/home/ubuntu/scripts/	CLI automation suite
Production folder	/home/ubuntu/ON_THE_BOULEVARD/	Everything operator-facing
Tracker	PRODUCTION_TRACKER.xlsx	5-sheet master status workbook

2. Getting Started

2.1 First-time setup

```
cd /home/ubuntu/scripts
bash setup.sh          # installs Python deps from requirements.txt
```

2.2 Verify the install

```
python3 signage_cli.py --help
```

You should see a list of sub-commands (`prompts` , `render` , `qc` , `export` , `status`).

2.3 Environment

- All paths are absolute under `/home/ubuntu/` .
- The Excel tracker can be edited locally; the system never overwrites it automatically.
- Logs are written to `/home/ubuntu/scripts/logs/` .

3. Workflow Step-by-Step

3.1 Add a new tenant

1. Open `/home/ubuntu/tenant_database.json` .
2. Append a new object to `tenants[]` with: `tenant_name` , `unit_number` , `category` , `facade_family` , `logo_file` , `illumination_type` , `notes` .

3. Drop the logo file into `/home/ubuntu/Shared/Uploads/` (or anywhere; just point `logo_file` to its absolute path).
4. Re-run the folder builder:

```
bash
python3 /home/ubuntu/build_otb.py
```

This creates `TENANTS/{UNIT}_{NAME}/` with the five standard subfolders and copies the logo into `logo_source/`.
5. Add the tenant as a row in `PRODUCTION_TRACKER.xlsx` → Master Tenant List.

3.2 Generate prompts for a tenant

```
cd /home/ubuntu/scripts
python3 signage_cli.py prompts --tenant "Lola Pink"
```

This stacks: `layer_1_global_style` + `layer_2_category_{category}` + `layer_3_scene_{scene}` + any `tenant_specific/{slug}.txt` overrides, and writes the combined prompts to `TENANTS/{UNIT}_{NAME}/prompts/`.

You'll typically want 4 prompts per tenant:

- `hero_storefront.txt` (scene: daytime)
- `dusk_variant.txt` (scene: dusk)
- `close_up.txt` (scene: close_up)
- `streetscape.txt` (scene: streetscape)

3.3 Run AI image generation

```
python3 signage_cli.py render --tenant "Lola Pink" --scene hero_storefront
```

Renders are saved into `TENANTS/{UNIT}_{NAME}/renders/` with a timestamped filename: `lola_pink_hero_storefront_YYYYMMDD_HHMMSS_v1.png`.

Batch all tenants:

```
python3 signage_cli.py render --all --scene hero_storefront
```

3.4 QC checks

```
python3 signage_cli.py qc --tenant "Lola Pink"
```

The script checks for:

- correct aspect ratio
- minimum resolution (≥ 2048 px on the long edge)
- file integrity
- presence of all 4 scene variants

Then log results in `PRODUCTION_TRACKER.xlsx` → QC Log.

3.5 Handle revisions

1. Move the rejected render to `TENANTS/{UNIT}_{NAME}/revisions/`.
2. Add a note to `tenant_database.json` (`notes` field) describing the issue.

- 3. Optionally add a tenant-specific override at `/home/ubuntu/prompt_library/tenant_specific/{slug}.txt` .
 - 4. Re-run the render command. Increment the `Revision Count` cell in the tracker.
- Approved renders move to `TENANTS/{UNIT}_{NAME}/finals/` .

3.6 Export for different use cases

```
# Leasing deck (one tenant)
python3 signage_cli.py export --tenant "Lola Pink" --target leasing

# Investor package (all tenants)
python3 signage_cli.py export --all --target investor

# Broker marketing pack
python3 signage_cli.py export --tenant "Lola Pink" --target broker
```

Outputs are written to `LEASING_DECKS/` or `PORTFOLIO_EXPORTS/` .

4. CLI Command Reference

Command	Purpose	Example
<code>prompts</code>	Build layered prompts	<code>signage_cli.py prompts --tenant "HotWorx"</code>
<code>render</code>	Generate AI images	<code>signage_cli.py render --tenant "HotWorx" --scene dusk_variant</code>
<code>qc</code>	Quality check	<code>signage_cli.py qc --all</code>
<code>export</code>	Package deliverables	<code>signage_cli.py export --all --target investor</code>
<code>status</code>	Print production state	<code>signage_cli.py status</code>

All commands support `--dry-run` (preview only) and `--verbose` .

5. Troubleshooting

Symptom	Likely cause	Fix
Tenant not found	Name mismatch with DB	Use the exact <code>tenant_name</code> from <code>tenant_database.json</code>
Logo not copied	<code>logo_file</code> path empty or wrong	Update DB, re-run <code>build_otb.py</code>
Render too dark / wrong illumination	Missing category layer	Confirm <code>category</code> in DB matches a <code>layer_2_category_*.txt</code> file
Wrong tenant style	No tenant override	Add <code>prompt_library/tenant_specific/{slug}.txt</code>
Tracker shows “NEEDED”	No logo on file	Acquire logo, place in <code>logo_source/</code> , re-run build
Permission denied	File ownership	<code>chmod -R u+rw /home/ubuntu/ON_THE_BOULEVARD</code>

Logs live in `/home/ubuntu/scripts/logs/` — open the most recent file first.

6. Best Practices

- **Naming:** tenant folders are `{UNIT}_{Tenant_Name}` with non-alphanumeric chars replaced by `_`. Never rename manually — re-run the builder.
- **Logos:** prefer PNG with transparent background, ≥ 1024 px on the short edge. SVG accepted in `logo_source/` for archival.
- **Renderers:** keep only approved finals in `finals/`. Everything else stays in `renders/` or `revisions/`.
- **DB hygiene:** `tenant_database.json` is the single source of truth. Never hand-edit the tracker for tenant metadata without also updating the DB.
- **Versioning:** leave timestamps in render filenames; never overwrite.
- **QC:** log every QC pass/fail in the tracker — auditability matters for investor decks.
- **Backups:** snapshot `/home/ubuntu/ON_THE_BOULEVARD/` and `tenant_database.json` before any large batch operation.

For a 5-minute onboarding, see `QUICKSTART.md`. For architecture, see `SYSTEM_ARCHITECTURE.md`.